DOMAIN_MAP_REVIEW.md

Comparison of the 18-domain map you laid out against the current `hotel_collection_schema.sql` (now v1.2). Every table you proposed is classified as:

- **KEEP** — already exists, already correct, untouched
- **MODIFY** — already exists, extended with new columns
- **ADD** — new table, added in v1.2
- **DEFER** — genuinely good idea, but out of scope for this MVP; not built now
- **DUPLICATE** — would represent a fact the schema already represents elsewhere; not added, existing table used instead

No table was removed. Nothing in v1.0/v1.1 broke.

Domain 1 — Platform / Configuration

Proposed table	Verdict	Notes
`countries`, `currencies`, `languages`	**KEEP**	Already present, unchanged.
`platform_settings`, `platform_media`	**DEFER**	Site branding (logo, colors, name, contact) is already a centralized *frontend* config-file decision from earlier in this project — putting it in the DB too would create two sources of truth for the same thing. Revisit only if a non-technical admin needs to edit branding without a deploy.
`translations`, `translation_keys`	**DEFER**	See Domain 17.
`notification_templates`, `notifications`, `notification_deliveries`	**DEFER**	See Domain 16.

Domain 2 — Hotel / Product

Proposed table	Verdict	Notes
`hotels`, `room_types`, `rooms`	**KEEP**	
`amenities`, `hotel_amenities`, `room_type_amenities`	**KEEP**	Added last round, already reusable across hotel + room type.
`experiences`, `restaurants`, `faqs`	**KEEP**	Added last round.

Proposed table	Verdict	Notes
`services`, `service_options`	**DUPLICATE**	This would model the same fact as `extras` (a bookable hotel add-on with a price). Two tables for one concept invites drift — e.g. an airport transfer being defined in both `extras` and `services` with different prices. If you later need *tiered options* per service (e.g. sedan vs. van transfer), that's a real reason to revisit — but it can be added as `extra_options` off the existing `extras` table rather than a parallel domain.
`restaurants` (optional per your note)	**KEEP**	Already added; harmless to keep even if the current hotel doesn't use it — an empty table costs nothing.

Domain 3 — Room Inventory / Availability ☆

Proposed table	Verdict	Notes
`rooms`	**KEEP**	(Cross-referenced from Domain 2 — physical inventory.)
`availability`	**KEEP**	Exactly the shape you described — room_type × date × total/sold/out_of_order/blocked, plus the `version` column added for optimistic locking. This was correct before and stays untouched.
`room_blocks`	**ADD**	New. Gives `availability.blocked` an actual reason (maintenance / owner use / event / other), a date range, and who created the block — instead of an unexplained integer.
`room_assignments`	**DEFER**	`reservation_rooms.room_id` already tracks *which* physical room is currently assigned to a reservation. A full assignment-history table (tracking every reassignment during a stay) is a real future feature, but not needed until room-swapping-mid-stay is common enough to need an audit trail.

Domain 4 — Pricing / Rate Management

Proposed table	Verdict	Notes
`rate_plans`, `room_type_rate_plans`, `rates`	**KEEP**	Already the exact Room Type → Rate Plan → date-specific Rate model you described.
`promotions`, `promotion_rules`, `promotion_eligible_room_types`, `promotion_eligible_rate_plans`	**KEEP**	
`tax_fee_types`	**KEEP**	Added last round.

Domain 5 — Search / Filtering

Proposed table	Verdict	Notes

Proposed table	Verdict	Notes
`filter_definitions`, `filter_options`	**DEFER**	Matches your own caveat — filter values are already derivable from `room_types`, `amenities`, `rates`, `availability`, `reviews`, `rate_plans`. Worth adding only if the filter *set itself* needs to be editable by staff without a deploy.
`saved_searches`	**DEFER**	Same bucket as "recently viewed" — client-side for now, per the earlier decision in this project.

Domain 6 — Client / Guest

Proposed table	Verdict	Notes
`guests`	**KEEP**	
`guest_preferences`	**DUPLICATE**	`guests.preferences` (free text) already covers this at MVP scope. A structured preferences table (pillow type, floor, dietary) is worth it once the backoffice needs to *query* on preferences, not just display them.
`guest_addresses`	**DUPLICATE**	Billing address is captured per-invoice instead (Domain 11) — a guest's billing address is a billing-event fact, not a permanent identity fact (they may bill differently next stay).
`guest_documents`	**DUPLICATE**	Already covered by `check_ins.id_document_reference` (a tokenized pointer, not the raw document) — that's the one place the platform actually needs a document reference.
`guest_contacts`	**DUPLICATE**	`guests.email`/`phone` already cover primary contact; a full secondary-contacts table is enterprise scope not currently justified.

Domain 7 — Users / Agents / Back Office

Proposed table	Verdict	Notes
`users`, `roles`, `permissions`, `role_permissions`, `user_roles`	**KEEP**	Already a solid RBAC foundation, confirmed against your description (Reservation Agent, Reception Agent, Hotel Administrator, etc. — not AI agents).
`audit_logs`	**KEEP**	See Domain 18.

Domain 8 — Reservations

Proposed table	Verdict	Notes

Proposed table	Verdict	Notes
`reservations`, `reservation_rooms`, `reservation_guests`, `reservation_extras`, `reservation_charges`	**KEEP**	Typed, separate tables preserved exactly as you argued for — no generic `reservation_details` collapse.
`reservation_services`	**DUPLICATE**	Same reasoning as `services` in Domain 2 — `reservation_extras` already covers a selected bookable add-on and snapshots its price at booking time.
`reservation_modifications`	**DEFER**	A full field-level diff of every edit is real audit tooling. `audit_logs` already records "reservation modified" at the coarse level an MVP needs; revisit if support/ops need to see exactly *which field* changed and from what value.
`reservation_status_history`	**ADD**	New. Lightweight append-only log of status transitions (pending → confirmed → checked_in → …) — cheap, and useful both for a guest-facing booking timeline and for support.

Domain 9 — Modification / Cancellation

Proposed table	Verdict	Notes
`reservation_cancellations`	**ADD**	New. Captures who cancelled, the reason, and — importantly — a *snapshot* of whether it was refundable and what penalty/refund applied, taken from the rate plan's structured cancellation fields (Domain 4) at the moment of cancellation. This means a later change to a rate plan's policy never rewrites what actually happened on a past cancellation.
`cancellation_reasons`	**ADD**	New. Small reusable reason catalog (`guest_changed_plans`, `found_cheaper`, …) referenced by `reservation_cancellations`.
`reservation_status_history`	**ADD**	(See Domain 8 — this table also carries the general lifecycle, cancellation is just one transition in it.)

Domain 10 — Payment

Proposed table	Verdict	Notes
`payments`, `payment_transactions`	**KEEP**	

Proposed table	Verdict	Notes
`refunds`	**DUPLICATE**	Already representable as a `payment_transactions` row with `transaction_type = 'refund'`. A separate `refunds` table would just be the same fact stored twice.

Domain 11 — Invoicing

Proposed table	Verdict	Notes
`invoices`	**ADD**	New. The financial document — what the guest owes — kept separate from `payments` (how they paid), exactly per your distinction. Billing name/address/country are snapshotted onto the invoice itself.
`invoice_items`	**ADD**	New. Line items (room nights, extras, taxes, fees, discounts) referencing `invoices`.

Domain 12 — Check-in / Stay

Proposed table	Verdict	Notes
`check_ins`	**MODIFY**	Added `checked_out_at TIMESTAMP` directly on the existing row.
`check_in_guests`	**DUPLICATE**	`check_ins` is already one row per `reservation_guest_id` — that's the same grain this table would add.
`room_assignments`	**DEFER**	See Domain 3.
`check_outs`	**DUPLICATE**	Folded into `check_ins.checked_out_at` instead of a parallel table — checkout is the other end of the same stay event, not a separate entity.

Domain 13 — Reviews / Ratings / Feedback

Proposed table	Verdict	Notes
`reviews`	**MODIFY**	Added `cleanliness_rating`, `location_rating`, `service_rating`, `value_rating` (all nullable 1–5) alongside the existing overall `rating` — gives you the multi-category breakdown you described without a join.
`ratings`, `feedback_categories`	**DUPLICATE**	Would model exactly what the four new columns above already model — a generic category/score table only earns its place once the category list itself needs to be dynamic (admin-configurable), which isn't a current requirement.
`review_responses`	**DUPLICATE**	Added as `response_text` / `responded_at` / `responded_by_user_id` directly on `reviews` instead — a hotel's reply is 1:1 with the review, so a join table adds no new relationship shape.

Proposed table	Verdict	Notes
`feedback`	**DEFER**	General (non-review) feedback capture wasn't part of the original PRD scope; revisit if there's an actual use case beyond reviews.

Domain 14 — Complaints

Proposed table	Verdict	Notes
`complaints`, `complaint_categories`, `complaint_messages`, `complaint_attachments`, `complaint_status_history`	**DEFER**	This is a full support-ticketing system — real enterprise scope. Front-desk staff can handle issues operationally today. Clean to bolt on later since nothing else in the schema references it.

Domain 15 — Loyalty

Proposed table	Verdict	Notes
`loyalty_accounts`, `loyalty_tiers`, `loyalty_points_transactions`, `loyalty_rewards`, `loyalty_redemptions`	**DEFER**	Matches the original platform overview's explicit non-goals list. Only worth building once a loyalty program is a real, funded initiative.

Domain 16 — Notifications

Proposed table	Verdict	Notes
`notification_templates`, `notifications`, `notification_deliveries`, `notification_preferences`	**DEFER**	For the MVP, booking confirmation / cancellation / payment-receipt emails can be sent directly by the application layer without a persisted log. Worth adding once you need to audit delivery status, support multiple channels (SMS/push), or let guests set channel preferences.

Domain 17 — Content / Translations

Proposed table	Verdict	Notes
`translation_keys`, `translations`, `localized_contents`	**DEFER**	Static UI strings already go through the frontend's i18n library (an existing project decision). Dynamic, DB-editable multilingual *content* (hotel/room descriptions in EN/FR/AR editable without a redeploy) is a real future need — most useful once there's more than one hotel or a non-technical content editor in the picture.

Domain 18 — Audit / System

Proposed table	Verdict	Notes
`audit_logs`	**KEEP**	
`system_logs`, `admin_actions`	**DUPLICATE**	`audit_logs`' actor/action/resource/result/metadata shape already covers both — a system-level technical log (errors, request logs) belongs in an observability tool (Domain 44 of the original platform overview), not the business database.

Summary

Verdict	Count
KEEP	32 tables
MODIFY	2 tables (`reviews`, `check_ins`)
ADD	6 tables (`room_blocks`, `reservation_status_history`, `cancellation_reasons`, `reservation_cancellations`, `invoices`, `invoice_items`)
DEFER	5 whole domains (Search/Filtering, Complaints, Loyalty, Notifications, Content/Translations) + a handful of individual tables in other domains
DUPLICATE	11 proposed tables — each mapped to an existing table that already represents the same fact

Net result: 39 → 45 tables. Every addition ties back to something concrete already in your requirements (structured cancellations, itemized taxes needing an actual invoice, a reason for `blocked` inventory) rather than speculative future-proofing — consistent with "practical for the MVP, room to expand."